

Mobile Tastatur 10-Tasten-Edition

Softwarepraxis & Vertiefungsprojekt IT, Fortsetzungsprojekt, Prof. Dr. rer. nat. Gerwinsiki

Lennart Langer

24. September 2026

Matr.-Nr. 018363935

Studiengang: KIA Angewandte Informatik
Hochschule Bochum - Campus Velbert Heiligenhaus

Inhaltsverzeichnis

1	Einleitung und Ausgangslage	3
1.1	Rückblick auf das Vorgängerprojekt	3
2	Zielsetzung	3
3	Hardware und Verdrahtung	3
3.1	Auswahl des Mikrocontrollers	3
3.2	Wegfall der Matrix	4
3.3	Tastenaufteilung auf beide Hände	4
3.4	Pinbelegung und Beschaltung	4
4	Software-Architektur	4
4.1	Grober Ablauf	4
4.2	Bitkodierung der Tastenzustände	5
4.3	Ringpuffer	5
5	Trigger und Entprellung	6
5.1	Zwei unabhängige Trigger	6
5.2	Arming/Disarming statt reiner Zeitmessung	6
5.3	Schutz vor Race Conditions	7
6	Eigene HID-Implementierung ohne Keyboard.h	7
6.1	Warum ein eigener HID-Report?	7
6.2	Selbstbeschreibung des Geräts	8
6.2.1	Der Anmeldeblock: Geräteart und Report-ID	8
6.2.2	Der eigentliche Report	8
6.2.3	Warum Reserved-Byte und genau sechs Keycode-Bytes?	9
6.2.4	Bekannte Einschränkung: Werte oberhalb von 101	9
6.3	Warum keine direkte Zeichenausgabe oder feste 101-Tasten-Tabelle?	9
6.4	Modifikatortasten als Sonderfall	10
6.5	Reguläre Tasten	10
7	Keymap und Beispiele	11
8	Vergleich zur Vorgängerversion	11
9	Fazit	12
10	Ausblick	12

1 Einleitung und Ausgangslage

1.1 Rückblick auf das Vorgängerprojekt

Im vorangegangenen Semester wurde im Rahmen des Projekts „Mobile Tastatur“ ein Handschuh-Prototyp entwickelt, der ohne Blickkontakt und ohne Touchscreen bedienbar sein sollte. Die Eingabe erfolgte dabei über 23 leitfähige Kontaktflächen aus Kupfergewebe, die an den Fingergliedern von Zeige-, Mittel-, Ring- und kleinem Finger der linken Hand angebracht waren. Über eine mit Potential belegte Kontaktfläche am Daumen wurden an diesen Flächen bei Berührung ein Stromkreis geschlossen. Die Auswertung erfolgte über eine 5×5-Matrixschaltung mit Dioden, die 25 mögliche Tastenkombinationen auf 10 Arduino-Anschlüssen abbildete. Die Ausgabe an den Host-Rechner lief über die offizielle **Arduino-Keyboard-Bibliothek**.

Dieser Ansatz funktionierte grundsätzlich, brachte aber mehrere Nachteile mit sich, die die Grundlage für die Weiterentwicklung in diesem Semester bilden:

- **Fehleranfälligkeit durch Matrixverschaltung:** Da mehrere Tasten dieselbe Zeile bzw. Spalte teilen, konnte es bei ungenauer Betätigung zu Queraktivierungen kommen.
- **Keine parallele Betätigung möglich:** Durch die Verschaltung der Matrix ist es nicht möglich sinnvoll das Drücken mehrerer Kupferfelder parallel auszulesen.
- **Abhängigkeit von der Propriozeption:** Die Erkennung, welches Fingerglied gerade berührt wird, setzt eine intakte Eigenwahrnehmung voraus, die nicht bei allen Personen gleich ausgeprägt ist.
- **Komplexer Aufbau:** Pro Kontakt wurden zwei Dioden benötigt, was den Handschuh mechanisch aufwendig und fehleranfällig in der Fertigung machte. Zudem sind die vorhandenen Leitungen anfällig für Zugbelastung, sodass sich eine Leitung an den Kontaktpunkten auf Dauer leicht lösen kann.
- **Abhängigkeit von der Keyboard-Bibliothek:** Die Ausgabe lief vollständig über die von Arduino bereitgestellte Bibliothek, wodurch der tatsächliche Aufbau eines HID-Reports im letzten Projekt nicht selbst entwickelt werden musste.

2 Zielsetzung

Für die Weiterentwicklung wurden folgende Ziele festgelegt:

- **Nutzung beider Hände,** um den informativen Anteil einer einzelnen Eingabe zu erhöhen und so gezielt auch die Eingabegeschwindigkeit zu steigern.
- **Verzicht auf die Bibliothek `Keyboard.h`:** Der **HID-Report-Deskriptor** soll eigenständig implementiert werden, um die tatsächliche Struktur eines **USB-HID**-Reports zu verstehen und volle Kontrolle über das gesendete Protokoll zu haben.
- **Umschaltbare Tastaturlayouts:** Das Gerät soll die Verwendung verschiedener Tastaturlayouts ermöglichen. Insbesondere soll zwischen einem deutschen und einem russischen Layout umgeschaltet werden können.
- **Weiterhin keine zusätzliche Software auf dem Host-System:** Das Gerät soll sich weiterhin als reines **USB-HID**-Gerät melden und ohne Treiberinstallation funktionieren („plug & play“).

3 Hardware und Verdrahtung

3.1 Auswahl des Mikrocontrollers

Die bereits im ersten Projektabschnitt getroffene Wahl des Arduino Micro wurde beibehalten, da dieser weiterhin eine für die neue Verdrahtung ausreichende Anzahl an Pins bietet. Unter den schon damals festgelegten Auswahlkriterien (native USB-Unterstützung, kompakte Baugröße, günstiger Preis) schneidet er weiterhin am besten ab.

3.2 Wegfall der Matrix

Die zentrale Änderung gegenüber dem Vorgängerprojekt ist der Verzicht auf die Matrixverschaltung. Jede der zehn Tasten besitzt nun eine eigene, direkte Leitung zu einem digitalen Eingang des Arduino Micro. Dadurch entfallen die zuvor notwendigen Dioden vollständig, und Queraktivierungen zwischen Tasten sind in der Schaltung selbst ausgeschlossen. Die Tasten können somit unabhängig voneinander ausgewertet werden. Dieser höhere Pinbedarf wird durch die Reduktion von 23 auf 10 Tasten jedoch mehr als ausgeglichen.

3.3 Tastenaufteilung auf beide Hände

Die zehn Tasten verteilen sich wie folgt:

- **8 Datentasten** (Pins 3–10), **je vier pro Hand**.
- **2 Triggertasten** (Pins 1 und 2), **je eine pro Hand**, die das Auslesen der Datentasten anstoßen.

Pro Hand ergeben sich damit fünf Tasten (vier Datentasten und eine Triggertaste), die elektrisch alle gegen ein gemeinsames **GND**-Kabel geführt werden. Jede der fünf Tasten besitzt eine eigene Signalleitung zum jeweiligen Arduino-Pin, nur die Rückleitung nach **GND** wird pro Hand gebündelt. Dadurch reduziert sich die Anzahl der Kabel, die zur Auswerteelektronik geführt werden müssen (5 Signalleitungen + 1 gemeinsame **GND**-Leitung pro Hand), ohne dass dabei eine Matrix mit den beschriebenen Nachteilen entsteht. Es handelt sich weiterhin um zehn elektrisch unabhängige Taster, aber nicht mehr um eine Kreuzverschaltung. Die Beschränkung auf acht Datentasten wurde gewählt, da sich damit exakt ein vollständiges Byte abbilden lässt, welches im HID-Report direkt als ein einzelner Keycode übertragen werden kann. Hierbei fungieren die Daumen als Bestätigungstasten (Trigger), die festlegen, wann genau dieses Byte aus den acht Datentasten ausgelesen und als Eingabe gewertet wird. Zusammen ergeben sich so die zehn Tasten der aktuellen Umsetzung. Eine höhere Tastenzahl würde zwar mehr Eingabemöglichkeiten schaffen, gleichzeitig aber den technischen Aufwand und die motorische Komplexität der Bedienung erhöhen. Die gewählte Tastenzahl stellt so einen Kompromiss zwischen der Anzahl möglicher Eingaben, dem technischen Aufwand und der Bedienbarkeit dar.

3.4 Pinbelegung und Beschaltung

Alle zehn Eingänge werden über interne **Pull-up-Widerstand** des Arduino Micro (`INPUT_PULLUP`) betrieben. Im Ruhezustand liegt an jedem Pin damit ein High-Pegel an. Ein Tastendruck verbindet den Pin mit dem gemeinsamen **GND** der jeweiligen Hand und zieht ihn so auf Low. Diese Logik muss folglich in der Software invertiert werden, um einen Tastendruck als „aktiv“ zu interpretieren.

Pin	Funktion	Bemerkung
1	Trigger 1	Interrupt, fallende Flanke
2	Trigger 2	Interrupt, fallende Flanke
3 – 6	Datentasten Hand A	je eigene Leitung
7 – 10	Datentasten Hand B	je eigene Leitung

Tabelle 1: Pinbelegung der 10-Tasten-Edition

4 Software-Architektur

4.1 Grober Ablauf

Der Ablauf orientiert sich am Prinzip des Vorgängerprojekts, wurde jedoch um einen zweiten unabhängigen Trigger erweitert und komplett auf eine eigene HID-Implementierung umgestellt:

1. Ein Tastendruck an Pin 1 oder Pin 2 löst über `attachInterrupt()` eine fallende Flanke aus und ruft die zugehörige Methode (`pressed()` bzw. `pressed2()`) auf.
2. Ist der jeweilige Trigger „scharf“ (armed), ruft die **ISR** `captureKeys()` auf.

3. `captureKeys()` liest über `readKeys()` den Zustand aller acht Datentasten als ein Byte aus und legt dieses Byte im [Ringpuffer](#) ab.
4. In der `loop()`-Funktion werden die im Puffer abgelegten Bytes nacheinander entnommen und an `processSnapshot()` übergeben, welche daraus einen vollständigen HID-Report baut und über `sendReport()` verschickt.

Damit bleibt die zeitkritische Arbeit (das reine Erfassen des Tastenzustands) in der [ISR](#) minimal, während die aufwendigere Umsetzung in ein HID-Signal im Hauptprogramm. Dieses Prinzip wurde schon im Vorgängerprojekt genutzt und hier auf zwei Trigger erweitert.

4.2 Bitkodierung der Tastenzustände

Die Funktion `readKeys()` liest die vier [PORT-Register](#) `PINB`, `PINC`, `PIND` und `PINE` aus und extrahiert daraus gezielt die acht Bits, die zu den Pins 3–10 gehören. Da diese Pins auf dem Arduino Micro nicht in einer fortlaufenden Reihenfolge auf den physischen Ports liegen, müssen die einzelnen Bits durch Maskierung und Schieben an die richtige Stelle in einem gemeinsamen Ergebnis-Byte gebracht werden:

```

1 uint8_t readKeys()
2 {
3     uint8_t d = PIND;
4     uint8_t c = PINC;
5     uint8_t e = PINE;
6     uint8_t b = PINB;
7
8     uint8_t result =
9         ((b & 0x70) << 1)
10        | (d & 0x01)
11        | ((d & 0x10) >> 3)
12        | ((c & 0x40) >> 4)
13        | ((d & 0x80) >> 4)
14        | ((e & 0x40) >> 2);
15
16     return ~result;
17 }

```

Das abschließende `~result` invertiert alle acht Bits, da die [Pull-up-Widerstand](#)-Beschaltung eine gedrückte Taste als Low-Pegel liefert. Das Ergebnis ist ein sogenanntes „Snapshot“-Byte, in dem jedes Bit für genau eine physische Taste steht. Dieser Wert wird als ein einziger Code ausgewertet, nicht als einzelne Tastendrücke:

Arduino-Pin	Bitwert im Snapshot
3	0x01
4	0x02
5	0x04
6	0x08
7	0x10
8	0x20
9	0x40
10	0x80

Tabelle 2: Bitzuordnung der Datentasten im Snapshot-Byte, angewendet im Registerzugriffe in `readKeys()`

4.3 Ringpuffer

Wie im Vorgängerprojekt wird ein [Ringpuffer](#) nach dem [FIFO](#)-Prinzip verwendet, hier mit 16 Speicherplätzen für einzelne Snapshot-Bytes:

```

1 volatile uint8_t keyBuffer[16];
2 volatile uint8_t writePos = 0;

```

```
3 volatile uint8_t readPos = 0;
```

Das Weiterzählen der Schreib- und Leseposition erfolgt über eine bitweise UND-Verknüpfung mit 15 ($((\text{writePos} + 1) \& 15)$). Bei einer Puffergröße von genau 16 Elementen ist dies gleichbedeutend mit der Modulo-Operation $\% 16$: Die Maske schneidet den Überlauf ab und springt so nach Index 15 wieder auf Index 0. Dies funktioniert nur, weil 16 eine Zweierpotenz ist. Die Maskierung wurde gewählt, weil sie bereits im Quellcode als einzelne Bitoperation formuliert ist und somit nicht vom Optimierungsverhalten des Compilers abhängt. In `captureKeys()` wird vor dem Schreiben geprüft, ob der Puffer voll ist (`next == readPos`), ist dies der Fall, wird der neue Snapshot verworfen, um ein Überschreiben noch nicht verarbeiteter Daten zu verhindern. Da stets ein Slot freigehalten wird, um „voll“ von „leer“ unterscheiden zu können, fasst der Puffer effektiv 15 Einträge. Da `keyBuffer`, `writePos` und `readPos` sowohl von der `ISR` als auch von `loop()` verwendet werden, sind sie als `volatile` deklariert. Dies stellt sicher, dass der Compiler Änderungen dieser Variablen durch die `ISR` berücksichtigt und Zugriffe darauf nicht unzulässig optimiert.

Im regulären Betrieb ist ein voller Puffer nicht zu erwarten: Die Verarbeitung eines Snapshots dauert nur wenige Millisekunden, während die Triggerlogik pro Trigger mindestens die Haltedauer des Tasters plus 10 ms Freigabezeit zwischen zwei Eingaben erzwingt. Der Puffer dient daher primär dazu, nahezu gleichzeitige Eingaben beider Hände sowie kurzzeitige Verzögerungen beim USB-Senden abzufangen.

Wichtige Einschränkung: Getrennte Verarbeitung Gepufferter Tasteneingaben. Der [Ringpuffer](#) sorgt zwar dafür, dass kein Tastendruck verloren geht, wenn kurz hintereinander mehrere Trigger Trigger auslösen. Er sorgt aber nicht dafür, dass daraus ein gemeinsamer, mehrere Tasten umfassender HID-Report entsteht. `loop()` entnimmt die Einträge strikt nacheinander: Für jeden einzelnen Snapshot wird in `processSnapshot()` ein vollständiger Zyklus aus Senden, 5 ms halten und wieder Loslassen durchlaufen (siehe Abschnitt 6.5), bevor der nächste Eintrag aus dem Puffer geholt wird. Werden also z. B. Trigger 1 und Trigger 2 fast gleichzeitig ausgelöst, landen zwar beide Snapshots sicher im Puffer, sie werden aber trotzdem als zwei getrennte, kurz nacheinander gesendete Reports mit jeweils nur `keys[0]` belegt ausgegeben, nicht als ein einziger Report mit `keys[0]` und `keys[1]`. Ein „echtes“ gleichzeitiges Mehrfach-Tippen mehrerer Tasten ist mit der aktuellen Verarbeitung folglich nicht möglich, unabhängig davon, wie schnell die Tasten tatsächlich gedrückt werden. Dies ist insbesondere für Tastenkombinationen relevant, bei denen neben mehreren Modifier-Tasten auch mehrere reguläre Tasten gleichzeitig gedrückt sein müssen. Eine Kombination wie **Strg+Shift+H+S** würde erfordern, dass sowohl **H** als auch **S** gleichzeitig in einem HID-Report enthalten sind. Da die regulären Tasten aktuell nacheinander verarbeitet werden, werden sie nicht als gleichzeitig gedrückte Tasten übertragen.

5 Trigger und Entprellung

5.1 Zwei unabhängige Trigger

Im Gegensatz zum Vorgängerprojekt, das mit einem einzelnen Trigger an Pin D2 arbeitete, nutzt die 10-Tasten-Edition zwei unabhängige Trigger an den Pins 1 und 2 – einen pro Hand:

```
1 pinMode(2, INPUT_PULLUP);
2 attachInterrupt(digitalPinToInterrupt(2), pressed2, FALLING);
3
4 pinMode(1, INPUT_PULLUP);
5 attachInterrupt(digitalPinToInterrupt(1), pressed, FALLING);
```

Beide Trigger lösen unabhängig voneinander eine Erfassung der acht Datentasten aus. Da jeder Trigger einen eigenen Arming-Zustand besitzt, kann der eine Trigger auslösen, auch wenn der andere noch nicht wieder scharf geschaltet ist

5.2 Arming/Disarming statt reiner Zeitmessung

Während im Vorgängerprojekt die Entprellung allein über einen Mindestabstand von 100 ms zwischen zwei Eingaben mit `millis()` realisiert wurde, verwendet dieses Projekt einen zustandsbasierten Mechanismus: Jeder Trigger befindet sich entweder im Zustand *armed* (bereit, einen neuen Tastendruck zu erkennen) oder *disarmed* (gesperrt, bis der Taster sicher wieder losgelassen wurde):

```

1 void pressed()
2 {
3     if (trigger1Armed)
4     {
5         trigger1Armed = false;
6         captureKeys();
7     }
8 }

```

Nach dem Auslösen wird der Trigger also sofort gesperrt. Erst wenn der zugehörige Pin für eine ununterbrochene Mindestdauer von 10 ms auf High liegt – der Taster also sicher losgelassen wurde und nicht mehr [Prellen](#) kann – wird der Trigger in `loop()` wieder scharf geschaltet:

```

1 if (!trigger1Armed)
2 {
3     if (digitalRead(1) == HIGH)
4     {
5         if (trigger1HighSince == 0)
6             trigger1HighSince = now;
7         else if ((uint32_t)(now - trigger1HighSince) >= RELEASE_TIME)
8         {
9             noInterrupts();
10            if (digitalRead(1) == HIGH)
11            {
12                trigger1Armed = true;
13                trigger1HighSince = 0;
14            }
15            interrupts();
16        }
17    }
18    else
19    {
20        trigger1HighSince = 0;
21    }
22 }

```

Fällt der Pin während der Wartezeit erneut auf Low (z. B. durch Prellen beim Loslassen), wird `trigger1HighSince` zurückgesetzt und die Wartezeit beginnt von vorn. Dieses Verfahren verhindert, dass ein einzelner physischer Tastendruck mehrfach als Eingabe gezählt wird, ohne dass wie im Vorgängerprojekt ein so großer Mindestabstand zwischen den Eingaben in Kauf genommen werden muss.

5.3 Schutz vor Race Conditions

Da `trigger1Armed` sowohl in der [ISR](#) `pressed()` als auch in `loop()` verändert wird, erfolgt die eigentliche Freigabe innerhalb eines kritischen Abschnitts (`noInterrupts()` / `interrupts()`) mit einer erneuten Prüfung des Pinzustands unmittelbar vor dem Setzen von `trigger1Armed = true`. Damit wird ausgeschlossen, dass zwischen der Prüfung in `loop()` und dem tatsächlichen Scharfschalten ein neuer Interrupt unbemerkt bleibt.

6 Eigene HID-Implementierung ohne Keyboard.h

6.1 Warum ein eigener HID-Report?

Ein zentrales Ziel und Teil der Aufgabenstellung des Projekts war es, nicht länger auf die von Arduino bereitgestellte `Keyboard`-Bibliothek zurückzugreifen, sondern den [HID-Report-Deskriptor](#) selbst zu definieren. Die Bibliothek kapselt den Aufbau des Reports vollständig hinter Funktionen wie `Keyboard.press()`, sodass Deskriptor, Report-ID und Byte-Layout für den Anwender unsichtbar bleiben. Ein eigener Report macht diese Struktur nachvollziehbar und gibt volle Kontrolle darüber, welche Bytes tatsächlich an den Host gesendet werden. Dazu wird direkt auf die HID-Kernbibliothek des Arduino-USB-Stacks zugegriffen und ein eigener [HID-Report-Deskriptor](#) angemeldet:

```

1 HIDSubDescriptor node(hidReportDescriptor, sizeof(hidReportDescriptor));
2
3 class KeyboardHID_ {
4 public:
5     KeyboardHID_() {
6         HID().AppendDescriptor(&node);
7     }
8 };

```

6.2 Selbstbeschreibung des Geräts

6.2.1 Der Anmeldeblock: Geräteart und Report-ID

Bevor festgelegt wird, wie ein einzelner Report aufgebaut ist, muss er dem Host zunächst mitteilen, um was für ein Gerät es sich überhaupt handelt. Dies übernehmen die ersten acht Bytes:

```

1 0x05, 0x01, // Usage Page (Generic Desktop Controls)
2 0x09, 0x06, // Usage (Keyboard)
3 0xA1, 0x01, // Collection (Application)
4 0x85, 0x01, // Report ID (1)

```

- **Usage Page (Generic Desktop Controls):** ordnet das Gerät zunächst grob der Kategorie „gewöhnliche Desktop-Eingabegeräte“ zu (zu der z.B. auch Maus und Joystick gehören).
- **Usage (Keyboard):** legt innerhalb dieser Kategorie konkret fest, dass es sich um eine Tastatur handelt. Genau dieses eine Feld ist es, das den Host dazu veranlasst, das Gerät wie eine normale Tastatur zu behandeln.
- **Collection (Application):** eröffnet eine zusammenhängende logische Gruppe, in der alle nachfolgenden Felder (Modifier-Bits und Keycode-Array) gebündelt werden. Sie wird am Ende durch 0xC0 wieder geschlossen.
- **Report ID (1):** Da ein einzelnes USB-HID-Interface grundsätzlich mehrere unterschiedlich aufgebaute Reports über denselben Kanal schicken könnte, bekommt jeder Report-Typ eine eindeutige Nummer vorangestellt. Dieses Projekt definiert nur einen einzigen Report-Typ mit der ID 1. Das ist exakt dieselbe Zahl, die `sendReport()` später bei jedem `HID().SendReport(1, ...)` mitschickt, damit der Host das gesendete Paket korrekt diesem Deskriptor zuordnen kann.

6.2.2 Der eigentliche Report

Nach dem Anmeldeblock folgt die Definition des Reports selbst. Er entspricht in seiner Struktur dem in der offiziellen [USB-HID-Spezifikation](#) für Tastaturen beschriebenen [Boot-Protokoll](#) und wird hier vollständig manuell als Byte-Array definiert. Inhaltlich beschreibt er drei Felder pro gesendetem Report:

- **1 Byte Modifier** – acht einzelne Bits für die Modifikatortasten (Strg, Umschalt, Alt, Windows/GUI, jeweils links und rechts).
- **1 Byte Reserved** – ungenutztes, konstantes Füllbyte, das laut Spezifikation vorgesehen, aber nicht belegt ist.
- **6 Byte Keycode-Array** – bis zu sechs gleichzeitig gedrückte reguläre Tasten. Jedes der sechs Bytes ist zwar als volles 8-Bit-Feld angelegt und könnte damit technisch jeden Wert von 0x00 bis 0xFF speichern, der Deskriptor deklariert über `Logical Minimum/Logical Maximum` sowie `Usage Minimum/Usage Maximum` jedoch ausdrücklich nur den Bereich 0 bis 0x65 (dezimal 101) als gültig – exakt der aus der USB-HID-Tabelle bekannte Wertebereich der Tasten eines klassischen 101-Tasten-Keyboards. Werte oberhalb von 101 sind laut Deskriptor nicht definiert. Der Host darf somit solche Werte ignorieren.

Diese Struktur wird in der `KeyReport`-Struktur nachgebildet, die exakt dessen Größe und Reihenfolge entspricht:

```

1 typedef struct {
2     uint8_t modifiers;
3     uint8_t reserved;
4     uint8_t keys[6];
5 } KeyReport;

```

Das eigentliche Versenden übernimmt `sendReport()`, das den kompletten Struct-Inhalt über die zuvor registrierte Report-ID 1 an den Host schickt:

```

1 void sendReport() {
2     HID().SendReport(1, &report, sizeof(report));
3 }

```

6.2.3 Warum Reserved-Byte und genau sechs Keycode-Bytes?

Die Gesamtstruktur von acht Byte (1 Byte Modifier, 1 Byte Reserved, 6 Byte Keycodes) folgt dem [Boot-Protokoll](#) der [USB-HID](#)-Spezifikation für Tastaturen. Einfache Umgebungen wie BIOS/UEFI oder Bootloader werten den individuellen Report-Deskriptor oft nicht aus und erwarten genau dieses Format. Ein verkürzter Report können dort fehlerhaft interpretiert werden, unter einem vollständigen Betriebssystem mit eigenem HID-Treiber aber weiterhin funktionieren, da dieser den Deskriptor korrekt ausliest. Da „Plug & Play ohne Zusatzsoftware“ zu den erklärten Zielen dieses Projekts gehört, wurde am Boot-Protokoll-Format festgehalten.

6.2.4 Bekannte Einschränkung: Werte oberhalb von 101

In `processSnapshot()` werden ausschließlich die acht Werte 0xE8 bis 0xEF explizit als Modifikator-Kommandos abgefangen (siehe Abschnitt 6.4). Alle übrigen Snapshot-Werte oberhalb von 101 (also die Bereiche 0x66 bis 0xE7 sowie 0xF0 bis 0xFF) werden dagegen unverändert in `keys[0]` geschrieben und als regulärer Report verschickt, obwohl sie außerhalb des vom Deskriptor deklarierten gültigen Bereichs (0 bis 101) liegen. In der Praxis ignorieren die meisten Betriebssysteme solche undefinierten Werte stillschweigend. Spezifikationskonform ist dieses Verhalten jedoch streng genommen nicht. Eine vollständige Lösung müsste zusätzlich alle Snapshot-Werte oberhalb von 101 gezielt abfangen oder das Tastenlayout von vornherein so wählen, dass ausschließlich gültige Kombinationen entstehen können.

6.3 Warum keine direkte Zeichenausgabe oder feste 101-Tasten-Tabelle?

Eine naheliegende Frage ist, warum die Tastatur nicht direkt Zeichen (z. B. ASCII) sendet oder eine feste Tabelle nach dem Vorbild klassischer 101-Tasten-Tastaturen (AT-101) verwendet.

Eine direkte Zeichenübertragung, etwa als ASCII oder in einer erweiterten Zeichenkodierung, scheidet aus. Zum einen sind Modifikatoren (Strg, Alt, Windows) sowie Pfeil- und Funktionstasten keine Zeichen; eine Tastenkombination wie Strg+C ließe sich in keiner Zeichenkodierung darstellen. Zum anderen behandelt das Betriebssystem nur Geräte mit dem Tastatur-Profil von USB-HID als Tastatur, und dieses überträgt Tastencodes statt Zeichen. Zwar ließen sich Zeichen auch über einen eigenen HID-Report oder eine virtuelle serielle Schnittstelle (USB-CDC) senden, doch müsste dann Zusatzsoftware auf dem Host diese Daten erst in Eingaben umsetzen. Das widerspricht dem Ziel, das Gerät ohne Treiber oder Zusatzsoftware zu betreiben (siehe Abschnitt 2). Würde die Firmware die Zeichen stattdessen selbst in Tastencodes zurückrechnen, wäre sie an ein festes Layout gebunden.

Klassische AT-Tastaturen übertragen keine Zeichen, sondern [Scancodes](#), die nur die Position der Taste angeben. Dieses Prinzip „Position statt Zeichen“ liegt auch USB-HID zugrunde. Die AT-Tastaturen übertragen ihre Scancodes jedoch über das PS/2-Protokoll. Dieser Anschluss ist an heutigen Hosts (z. B. Laptops, Tablets) oft nicht mehr vorhanden. [USB-HID](#) überträgt positionsbezogene Usage-Codes vollständig über USB. So steht beispielsweise 0x04 in den HID Usage Tables (Keyboard/Keypad Page) für „Keyboard a and A“. Welches Zeichen daraus entsteht, entscheidet erst der Host anhand des eingestellten Tastaturlayouts. Dieselbe Firmware funktioniert dadurch mit allen Layouts, und eine eigene Zeichentabelle ist nicht erforderlich.

Da die Tastatur selbst kein Sprachkonzept besitzt, lassen sich auch russische Zeichen ohne Codeänderung ausgeben, sofern am Host ein russisches Layout aktiv ist. Damit ist das in Abschnitt 2 formulierte Ziel erfüllt, ohne dass zusätzliche Software oder eine eigene Layoutlogik in der Firmware nötig wäre. Lediglich die Zuordnung von Fingerkombination zu Taste müsste sich der Nutzer für dieses Layout neu einprägen.

6.4 Modifikatortasten als Sonderfall

Da die acht Datentasten nur 256 mögliche Snapshot-Werte liefern können und ein Teil davon für reguläre Zeichen benötigt wird, werden acht Werte im oberen Bereich (0xE8 bis 0xEF) als reservierter Bereich für Modifikatortasten genutzt:

```
1 #define CMD_TOGGLE_LCTRL 0xE8
2 #define CMD_TOGGLE_LSHIFT 0xE9
3 #define CMD_TOGGLE_LALT 0xEA
4 #define CMD_TOGGLE_LGUI 0xEB
5 #define CMD_TOGGLE_RCTRL 0xEC
6 #define CMD_TOGGLE_RSHIFT 0xED
7 #define CMD_TOGGLE_RALT 0xEE
8 #define CMD_TOGGLE_RGUI 0xEF
```

Warum ausgerechnet mit diesen acht Werten? Bei der Umsetzung wurde eine Alternative bewusst verworfen: Statt den fest reservierten Kombinationswerte könnte man auch einen einzelnen „Escape“-Signal definieren, auf den dann ein zweiter, separat ausgewerteter Snapshot als eigentliche Modifikator-Auswahl folgt. Dieser zweistufige Ansatz wurde verworfen, weil er für jede Modifikatoraktivierung zwei getrennte, nacheinander ausgelöste Trigger-Ereignisse benötigt hätte, also zwei getrennte Tastendrucke statt eines einzigen. Dies hat nach dem eigenen Ermessen den Schreibfluss unterbrochen, und wurde deshalb verworfen. Stattdessen wurde ein einstufiges Verfahren gewählt. Die acht Werte von 0xE8 bis 0xEF werden fest den acht Modifikatortasten zugeordnet. Diese Werte liegen am oberen Ende des Wertebereichs und werden daher nicht für reguläre Zeichen verwendet. Eine entsprechende Eingabekombination löst dadurch unmittelbar die zugehörige Modifikatorfunktion aus, ohne dass ein zusätzliches Signal erforderlich ist. Wird ein reservierter Snapshot erkannt, wird das zugehörige Bit in der globalen Variable `heldModifiers` per XOR umgeschaltet (getoggelt), statt wie bei einer normalen Taste einen einzelnen Report zu senden und die Taste sofort wieder loszulassen:

```
1 if (modifier != 0)
2 {
3     heldModifiers ^= modifier;
4     leere();
5     report.modifiers = heldModifiers;
6     sendReport();
7     return;
8 }
```

Dadurch bleibt eine Modifikatortaste (z.B. Strg) so lange „gedrückt“, bis die zugehörige Kombination erneut eingegeben wird, um mit den vorhandenen zehn Tasten dennoch Tastenkombinationen wie Strg+C nachbilden zu können, ohne eine zusätzliche physische Halteoption zu benötigen. Gleichzeitig kann eine aktivierte Modifikatortaste auch ohne weitere reguläre Taste übertragen werden, indem ein leerer Snapshot (0x00) verarbeitet wird. Dadurch sind beispielsweise auch einzelne Betätigungen der Windows-Taste möglich.

6.5 Reguläre Tasten

Alle Snapshot-Werte außerhalb des reservierten Modifikator-Bereichs werden als normale Zeichen behandelt. Dabei wird der HID-Report zunächst mit dem gedrückten Keycode gesendet. Anschließend wird eine feste Haltezeit von 5 ms eingehalten, damit der Tastendruck für den Host mindestens über ein Abfrageintervall hinaus sichtbar bleibt und auch von Anwendungen erkannt wird, die den Tastenzustand selbst abfragen. Danach wird der Report wieder geleert und erneut gesendet, um die Taste logisch loszulassen:

```
1 leere();
2 report.modifiers = heldModifiers;
3 report.keys[0] = snapshot;
4 sendReport();
5
6 delay(5);
7 leere();
8 report.modifiers = heldModifiers;
```

```
9 sendReport();
```

Aktuell wird dabei stets nur `keys[0]` des sechs Byte großen Arrays belegt, auch wenn der Deskriptor bis zu sechs gleichzeitige Tasten erlauben würde. Wie in Abschnitt 4.3 beschrieben, liegt das nicht nur an dieser bewussten Vereinfachung, sondern auch daran, dass die aktuelle Puffer-Verarbeitung ohnehin jeden Eintrag einzeln und vollständig abarbeitet, bevor der nächste an der Reihe ist – eine Erweiterung auf mehrere Slots würde daher zusätzlich eine Änderung der Verarbeitungslogik selbst erfordern (siehe Abschnitt 10).

7 Keymap und Beispiele

Aus der Bitzuordnung der Datentasten lassen sich sowohl einzelne Tastendrucke als auch Kombinationen mehrerer gleichzeitig gedrückter Tasten als eindeutige Snapshot-Werte ableiten. Tabelle 3 zeigt eine Auswahl:

Gedrückte Pin(s)	Snapshot-Byte	Ergebnis
Pin 5	0x04	a
Pin 6	0x08	e
Pin 3 + Pin 6	0x09	f
Pin 4 + Pin 8	0x22	5
Pin 7 + Pin 8	0x30	+
Pin 6 + 8 + 9 + 10	0xE8	Strg (Modifikator)

Tabelle 3: Beispielhafte Keymap-Einträge für eine deutsche QWERTZ-Tastaturbelegung

Da die tatsächlich gesendeten Werte reine [USB-HID-Keycodes](#) sind (z. B. 0x04 für die Taste „A“), entspricht das dargestellte Zeichen stets der Interpretation dieses Keycodes durch das aktuell auf dem Host-System eingestellte Tastaturlayout, nicht zwingend fest dem hier notierten Zeichen.

Beispiel: Ein großes „A“ schreiben. Ein Großbuchstabe entsteht nicht aus einem einzelnen Snapshot, sondern aus zwei nacheinander ausgelösten Eingaben. Zunächst wird die Kombination 0xE9 (`CMD_TOGGLE_LSHIFT`) ausgelöst: Das Bit `MOD_LSHIFT` wird in `heldModifiers` gesetzt, und es wird sofort ein Report mit aktivem Shift-Modifizier, aber ohne gedrückte Zeichentaste verschickt. Anschließend erzeugt die reguläre Betätigung von Pin 5 (Snapshot 0x04) einen zweiten Report, dessen `modifiers`-Feld weiterhin das gesetzte Shift-Bit enthält und dessen `keys[0]` auf 0x04 steht. Der Host interpretiert diese Kombination aus Usage-Code 0x04 und aktivem Shift-Modifizier als „A“. Da `CMD_TOGGLE_LSHIFT` als Toggle implementiert ist, bleibt Shift so lange aktiv, bis dieselbe Kombination erneut betätigt wird – für ein einzelnes großes Zeichen muss der Modifizier also anschließend bewusst wieder deaktiviert werden, um nicht versehentlich weitere Zeichen großzuschreiben.

8 Vergleich zur Vorgängerversion

Merkmal	Vorher (1. Semester)	Jetzt (2. Semester)
Tasten	23 Kontaktflächen	10 (8 Daten + 2 Trigger)
Verdrahtung	5×5-Matrix mit Dioden	Direkter Input je Taste
Sensorik	Propriozeption, Berührungspunkte	Feste Taster
Hände	Nur linke Hand	Beide Hände
Fehleranfälligkeit	Queraktivierung möglich	Deutlich reduziert
Ausgabe	Arduino-Keyboard-Bibliothek	Eigener HID-Report-Deskriptor

Tabelle 4: Gegenüberstellung der beiden Projektphasen

Die konsequente Reduktion der Fehlerquellen und der Verzicht auf die Matrixschaltung gehen mit einer direkteren HID-Übergabe einher: Da nicht mehr über die `Keyboard`-Bibliothek, sondern über einen eigenen, minimalen Report gearbeitet wird, entfallen komfortable Befehle wie `Keyboard.press()`, wie sie im Vorgängerprojekt genutzt wurden. Modifikatortasten mussten dafür manuell über den reservierten Wertebereich 0xE8–0xEF nachgebildet werden. Zudem ist mit der aktuellen Implementierung kein Layoutwechsel

und keine Befehlstaste vorgesehen. Das Vorgängerprojekt konnte über die Variable `active` noch zwischen mehreren Tastaturebenen umschalten.

9 Fazit

Im Rahmen dieses zweiten Projektabschnitts konnte gezeigt werden, dass sich durch den Wegfall der Matrixverschaltung und die Aufteilung auf beide Hände die Fehleranfälligkeit der mobilen Tastatur deutlich reduzieren lässt, ohne auf das Grundprinzip einer softwarelosen, blind bedienbaren USB-Tastatur zu verzichten. Der zentrale technische Fortschritt liegt jedoch im eigenständig implementierten [HID-Report-Deskriptor](#): Statt sich auf die `Keyboard`-Bibliothek zu verlassen, wurde ein vollständiger, eigener HID-Report-Deskriptor entworfen, der Modifikatortasten über einen reservierten Wertebereich sowie reguläre Zeichentasten über direkte, standardisierte USB-HID-Positionscode abbildet. Damit folgt er bewusst demselben Grundprinzip, das bereits klassische Tastaturen nutzten. In Kombination mit dem bewährten [Ringpuffer](#)-Prinzip aus dem Vorgängerprojekt und einem neuen, zustandsbasierten Arming/Disarming-Mechanismus zur Entprellung zweier unabhängiger Trigger konnte eine robuste und gleichzeitig deutlich einfacher aufgebaute Eingabelösung realisiert werden.

10 Ausblick

Für eine mögliche weitere Ausbaustufe bieten sich folgende Anknüpfungspunkte an:

- **Nutzung mehrerer Keycode-Slots:** Der HID-Report unterstützt bereits sechs gleichzeitig übertragbare Tasten (`keys[0]` bis `keys[5]`). Aktuell wird jedoch nur `keys[0]` befüllt. Eine mögliche Erweiterung wäre, zukünftig zu prüfen, ob sich mehrere Einträge im [Ringpuffer](#) befinden, und diese bei Bedarf gemeinsam in einem HID-Report zu übertragen.
- **Einhändiger Betrieb:** Aktuell ist die Eingabe auf den gleichzeitigen Einsatz beider Hände ausgelegt; ein Zusatzmodus für nur eine verfügbare Hand ist bislang nicht vorgesehen.
- **Ergonomisches Gehäuse:** Die aktuelle Umsetzung besteht aus einer in der Hand gehaltenen Röhre mit zehn Tastern. Eine mögliche Weiterentwicklung wäre eine ergonomisch optimierte Gehäuseform, bei der die Taster gut erreichbar sind und das Gerät sicher und komfortabel in der Hand liegt.
- **Makros und Sonderfunktionen:** Der aktuell nicht für reguläre Zeichen verwendete Wertebereich könnte gezielt für Sonderfunktionen genutzt werden. Statt ausschließlich den Zustand einzelner Modifikatortasten über `heldModifiers` zu verändern, könnte `processSnapshot()` für bestimmte Snapshot-Werte auch vordefinierte Folgen von HID-Reports versenden. Dadurch könnte beispielsweise eine einzelne Eingabe das automatische Senden einer Zeichenfolge wie „Hello World“ auslösen.
- **Erweiterter HID-Report:** Alternativ könnten die bisher nicht für reguläre Zeichen verwendeten Snapshot-Werte über einen zweiten HID-Report an den Host übertragen werden. Speziell entwickelte Anwendungen könnten diese Eingaben unabhängig vom normalen Tastatur-Report auswerten und darauf basierend eigene Funktionen ausführen, die über die normale Tastatureingabe hinausgehen.

Hinweise

Da die Zeichenausgabe über reine [USB-HID](#)-Positionscode erfolgt, bestimmt ausschließlich das auf dem Host-System eingestellte Tastaturlayout, welches Zeichen tatsächlich erscheint. Ein und derselbe Snapshot-Wert kann auf einem System mit deutschem Layout ein anderes Zeichen erzeugen als auf einem System mit US- oder russischem Layout.

Die verwendete HID-Kernbibliothek basiert auf der offiziellen Arduino-Dokumentation <https://github.com/arduino-libraries/Keyboard/blob/master/src/Keyboard.h>, deren `Keyboard.h`-Abstraktion in diesem Projekt bewusst nicht mehr genutzt wird.

Die Zuordnung der Keycodes orientiert sich an den offiziellen USB-HID Usage Tables (Keyboard/Keypad Page): <https://www.usb.org/document-library/hid-usage-tables-14>.

Details zur Pinbelegung und den PORT-Registern des Arduino Micro: <https://docs.arduino.cc/hardware/micro/>.

Das vorherige Projekt ist auffindbar unter <https://github.com/Crafter04/Handkey/>.

Der für die 10-Tasten-Edition entwickelte Code ist auffindbar unter <https://github.com/Crafter04/Keyboard/>.

Glossar

Boot-Protokoll Ein in der USB-HID-Spezifikation fest vorgegebenes, stark vereinfachtes Report-Format für Tastaturen und Mäuse, das auch einfache Umgebungen wie ein BIOS/UEFI ohne Kenntnis des individuellen HID-Report-Deskriptors korrekt interpretieren können. 8, 9

FIFO First In, First Out, ein Prinzip zur Verwaltung von Daten in einer Warteschlange. 5

GND Masse: Bezugspotential der Schaltung (0 V). Ein Stromkreis ist erst geschlossen, wenn er zu GND zurückführt; beim Arduino ist GND der Minuspol der Spannungsversorgung. 4

HID-Report-Deskriptor Eine vom USB-HID-Gerät beim Verbindungsaufbau übermittelte Beschreibung, die dem Host-Betriebssystem mitteilt, wie die nachfolgend gesendeten Datenpakete (Reports) aufgebaut sind und zu interpretieren sind. 3, 7, 12

ISR Interrupt Service Routine, eine Funktion, die beim Auslösen eines Hardware-Interrupts automatisch ausgeführt wird. 4–7

PORT-Register Register eines Mikrocontrollers (z. B. `PINB`, `PIND`), über das der aktuelle logische Zustand mehrerer Pins gleichzeitig als ein Byte ausgelesen werden kann. 5

Prellen Mechanisches Nachschwingen eines Tasterkontakts beim Schließen oder Öffnen, wodurch kurzzeitig mehrere elektrische Impulse statt eines einzelnen Signals entstehen. 7

Propriozeption Wahrnehmung der eigenen Körperposition und Bewegung, z. B. Lage der Finger im Raum. 3

Pull-up-Widerstand Ein Widerstand, der einen Eingang im unbetätigten Zustand definiert auf ein High-Signal zieht; beim Arduino intern über `INPUT_PULLUP` aktivierbar, sodass ein Tastendruck den Pin stattdessen auf Low zieht. 4, 5

Ringpuffer Eine Datenstruktur, die Elemente in einem kreisförmigen Puffer speichert und wiedergibt. Ist der Puffer voll, werden je nach Implementierung neue oder alte Einträge verworfen. 5, 6, 12

Scancode Ein von einer Tastatur übertragener Code, der die physische Position einer gedrückten Taste beschreibt – unabhängig davon, welches Zeichen auf der Tastenkappe aufgedruckt ist oder welches Zeichen das Betriebssystem letztlich daraus erzeugt. 9

USB-HID Universal Serial Bus – Human Interface Device, ein Standard für Eingabegeräte wie Tastaturen oder Mäuse. 3, 8, 9, 11, 12